

LLFS

Linked List File System

for low volume media up to 64 KB

USER MANUAL

Version 1.1 · 26 August 2026

Written by Sergey Sanders

sergey@sesadesign.com

Part of ucosR, the firmware of the Rimer single board computer

Rimer SBC · Microchip SAMD51 with I²C EEPROM

Sesa Design Inc.

Copyright © 2025 Sergey Sanders. Released under the MIT License.

Revision history

Version	Date	Author	Description
1.0	19 July 2016	Sergey Sanders	First release of the module.
1.1	26 August 2026	Sergey Sanders	Index chain, write path, size accounting and name handling corrected. Sector 0xFF formally reserved. Host unit test added.

Contents

1 Introduction

- 1.1 Purpose
- 1.2 Design goals
- 1.3 Specifications
- 1.4 Source files

2 Media organisation

- 2.1 Volume partitioning
- 2.2 Index sector
- 2.3 Volume descriptor — `lf_phy_t`
- 2.4 File record — `lf_record_t`
- 2.5 The INDEX record
- 2.6 Data sector
- 2.7 Sector link encoding
- 2.8 Example: how a 600 byte file is stored
- 2.9 Free space accounting

3 Names, attributes and modes

- 3.1 File names
- 3.2 Wildcards
- 3.3 Attributes
- 3.4 Open modes
- 3.5 Error codes

4 API reference

- 4.1 `lf_init`
- 4.2 `lf_format`
- 4.3 `lf_get_free`
- 4.4 `lf_get_fsize`
- 4.5 `lf_find_record`
- 4.6 `lf_rname_tostr`
- 4.7 `lf_delete`
- 4.8 `lf_open`
- 4.9 `lf_close`
- 4.10 `lf_read`
- 4.11 `lf_write`

4.12 If_gets

5 Worked examples

- 5.1 Format the volume on first start
- 5.2 Save and load a configuration structure
- 5.3 List the volume
- 5.4 Append to a log file
- 5.5 Run a script line by line

6 Porting LLFS to another media

- 6.1 The volume descriptor
- 6.2 Driver callbacks
- 6.3 EEPROM driver

7 Limits and design notes

8 Verification

9 License

1 Introduction

1.1 Purpose

LLFS is a small flat file system for serial memory devices of up to 64 KB, such as the I²C EEPROM of the Rimer SBC. It stores a file as a linked list of 256 byte sectors: each sector carries the number of the sector that follows it, so no allocation table has to be held in RAM and no sector has to be rewritten when a file grows.

The whole module needs one 256 byte scratch buffer per volume plus one buffer per open file, which makes it usable on a controller with a few kilobytes of RAM.

1.2 Design goals

- No RAM resident allocation table: the free list lives in the last two bytes of every sector.
- Appending to a file never rewrites the sectors already stored.
- A single sector write is the largest atomic operation, so an interrupted write truncates a file instead of corrupting the volume.
- The directory is a chain of index sectors that grows on demand.
- The media driver is a table of five function pointers, so the format is not tied to EEPROM.

1.3 Specifications

Item	Value
Maximum volume size	65 536 bytes (256 sectors)
Sector size	256 bytes
Data bytes per full sector	255
Data bytes per partially filled sector	254 maximum
Sectors available for file data	254 (0x01 ... 0xFE)
Index sectors	sector 0x00, then any free sector; 0xFF is reserved for the second one
File records per index sector	15 (14 in sector 0, record 0 holds the volume descriptor)
Maximum file name length	12 characters, no terminator when all 12 are used
Maximum volume name length	8 characters
Maximum file size	64 770 bytes (the whole data area)
Maximum number of files	limited by free sectors: 239 single sector files on a 64 KB volume
Directories	record fields are reserved, not implemented
Concurrency	single task, see section 7

1.4 Source files

File	Contents
llfs.h	Public API, on media structures, attribute and mode constants.
llfs.c	The file system itself.
llfs_vol_eeeprom.h / .c	Volume driver for the I ² C EEPROM of the Rimer SBC.
llfs_unittest.c	Host unit test, builds and runs on the development machine.
llfs.md	Short format summary kept next to the sources.

2 Media organisation

The drawings in this section use three colours throughout:

Blue	Index sectors and the records they hold: the directory of the volume.
Yellow	File data, and the COUNT byte that says how much of a sector is valid.
Red	The LINK byte and the reserved sector 0xFF: the chain machinery, never to be overwritten by hand.

2.1 Volume partitioning

A volume is a flat array of 256 byte sectors. Sector 0 is always the first index sector. Sector 0xFF is never used for file data, because the value 0xFF in a sector link means “end of chain” and could not be told apart from a reference to that sector; it is held back for the second index sector.

0x00	0x01 ... 0xFE	0xFF
Index[0]	data sectors, or a further index sector	Index[1]

Sector	Type	Purpose
0x00	Index[0]	Volume descriptor, 14 file records and the INDEX record.
0x01 ... 0xFE	Data	File data, 255 bytes each, or an additional index sector.
0xFF	Index[1]	Reserved: claimed by the second index sector, never file data.

2.2 Index sector

Every index sector holds sixteen 16 byte records. The last one is the INDEX record: it links to the next index sector and carries the marks that identify the sector. In sector 0 the first record is replaced by the volume descriptor.

0x00 – 0x0F	0x10 – 0xEF	0xF0 – 0xFF
lf_phy_t	lf_record_t × 14	INDEX lf_record_t
volume descriptor (sector 0 only)	file records	link to the next index sector

Sector 0 only

In every other index sector the first record is an ordinary file record, so those sectors hold 15 files instead of 14.

2.3 Volume descriptor — `lf_phy_t`

Sixteen bytes at offset 0x00 of sector 0.

Offset	Size	Field	Type	Description
0x00	1	fsType	uint8_t	0xDB when the volume is formatted.
0x01	1	fsTypeN	uint8_t	One's complement of fsType, 0x24.
0x02	2	devID	uint16_t	Device identifier, 0x0400 for the EEPROM.
0x04	2	secCount	uint16_t	Number of sectors in the volume, 256 for 64 KB.
0x06	1	compress	uint8_t	Compression, LLFS_COMPRESS_NONE.
0x07	1	ecc	uint8_t	ECC base size, LLFS_ECC_NONE.
0x08	8	name	uint8_t[8]	Volume name, no terminator when all 8 are used.

2.4 File record — `lf_record_t`

Sixteen bytes. A record whose name starts with 0xFF is an erased, free slot.

Offset	Size	Field	Type	Description
0x00	1	fptr	uint8_t	First data sector of the file, 0xFF for a directory or a link.
0x01	1	attr	uint8_t	Attribute bits, see section 3.3.
0x02	2	rptra	uint16_t	Reference to the parent directory, 0 for the root.
0x04	12	name	char[12]	File name, terminated only when shorter than 12.

2.5 The INDEX record

The last record of an index sector, at offset 0xF0, overlaps the two bytes that every sector uses for its link. That overlap is deliberate but it constrains what may be written there.

0xF0	0xF1	0xF2 – 0xF3	0xF4 – 0xFD	0xFE	0xFF
fptr	attr	rptr	name[0..9]	name[10]	name[11]
next index sector, 0x00 if last	0x40 INDEX	unused	unusable	must stay non zero	must stay 0x00

Why the last two bytes matter

Byte 0xFF is the index sector mark and must read 0x00. Byte 0xFE is the same byte a data sector uses for its valid byte count: if both were zero the sector would look free and could be handed out for file data, destroying the directory. The name field of the INDEX record is therefore not usable.

2.6 Data sector

The first 254 bytes always hold data. The last two bytes describe the sector: how much of it is valid and where the file continues.

0x00 ... 0xFD	0xFE	0xFF
254 data bytes	COUNT	LINK
always data	valid bytes, or the 255th data byte when the sector is full	next sector, self, or 0xFF

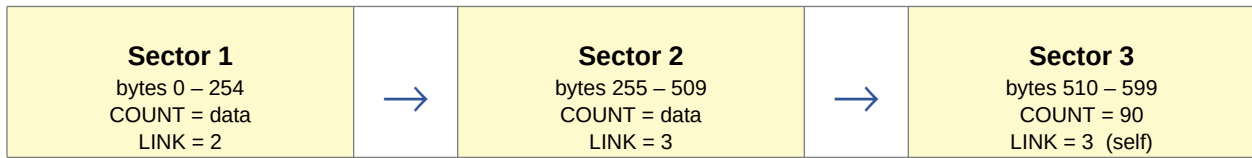
2.7 Sector link encoding

COUNT and LINK are read and written together as one 16 bit little endian value, LINK in the high byte. The four states of a sector are:

LINK	COUNT	State	Meaning
0x00	0x00	Free	The sector is not allocated, <code>lf_get_free_sector()</code> may hand it out.
self	0 ... 254	Partially filled	The sector is the last of its file and holds COUNT valid bytes.
n	data	Full, chained	All 255 bytes are data, the file continues in sector n. COUNT holds the 255th data byte.
0xFF	data	Full, last	All 255 bytes are data and the file ends here.

2.8 Example: how a 600 byte file is stored

The file record points at sector 1. Two full sectors carry 255 bytes each, the third one is partially filled and closes the chain by pointing at itself.



If `lf_get_fsize()` walks the same chain: 255 + 255 for the two full sectors, then 90 from the COUNT byte of the last one, which gives 600.

2.9 Free space accounting

If `lf_get_free()` counts every sector between 1 and `secCount-2` whose link word is zero and reports 255 bytes for each. A freshly formatted 64 KB volume therefore reports 64 770 bytes, which is 254 sectors of 255 bytes. Index sectors are allocated from the same pool and are never released, so a volume that has held many files at once keeps those sectors for later records.

3 Names, attributes and modes

3.1 File names

A name occupies up to 12 characters in the record. A shorter name is followed by a zero, a name of exactly 12 characters is not terminated at all, so a record name must never be passed to a string function directly. `lf_name_tostr()` is the only safe way to read one: it copies at most 12 characters into a buffer of 13 bytes and always terminates the result.

```
char str[LLFS_FILENAME_LEN + 1];      // 13 bytes, always
lf_record_t record;

if (lf_find_record("conf.sys", &record, 0))
    tprintf("%s\n", lf_name_tostr(str, record.name));
```

A name longer than 12 characters passed to `lf_open()` is truncated, both in the record and in `lf_t::name`, so the file can be found again with the truncated name.

3.2 Wildcards

`lf_find_record()` and `lf_delete()` accept an expression instead of a plain name. The only wildcard is the asterisk, which stands for any number of characters, the empty string included. Matching is case sensitive.

Expression	Matches	Does not match
*	every file	—
*.sys	conf.sys	conf.cfg
conf*	conf.sys, conf.bak	myconf
conf	conf.sys, myconf.old	data.bin
*s	conf.sys	conf.cfg
c*.sys	conf.sys, c.sys	path.sys

3.3 Attributes

Constant	Value	Meaning
LLFS_ATTR_READ	0x01	The file may be read.
LLFS_ATTR_WRITE	0x02	The file may be written.
LLFS_ATTR_EXEC	0x04	The file is executable, a shell script.
LLFS_ATTR_LOCK	0x08	Reserved.
LLFS_ATTR_DIR	0x10	Directory. Reserved, not implemented.
LLFS_ATTR_LINK	0x20	Link. Reserved, not implemented.
LLFS_ATTR_INDEX	0x40	System: fptr points to the next index sector.
LLFS_ATTR_FVALID	0x80	System: the record is a valid file descriptor.

Modes and attributes are not the same bits

MODE_EXEC is 0x01 while LLFS_ATTR_EXEC is 0x04, and MODE_READ is 0x04 while LLFS_ATTR_READ is 0x01. Use the LLFS_ATTR_FROM_MODE() macro to convert, never a plain mask of the mode.

3.4 Open modes

Constant	Value	Meaning
MODE_EXEC	0x01	Marks the created file as executable.
MODE_WRITE	0x02	Open for writing. Without MODE_APPEND the file is truncated.
MODE_READ	0x04	Open for reading.
MODE_CREATE	0x08	Create the file if it does not exist. Combine with MODE_WRITE.
MODE_APPEND	0x10	Start writing at the end of the file. Requires MODE_WRITE.

The mode decides the stored attributes

The attributes of a new file come from the mode it was created with, so a file created with MODE_CREATE | MODE_WRITE is listed as “-w-”. Add MODE_READ, or MODE_EXEC for a script, when the record should carry those rights.

3.5 Error codes

Every call reports through the global `lf_error`. The media driver sets it on each access, so read it immediately after the call that failed.

Code	Value	Raised when
LF_ERR_NONE	0	The operation succeeded.
LF_ERR_VOLUME	1	The sector read is not a valid index sector, the volume is not formatted.
LF_ERR_READ	2	The media driver failed to read.
LF_ERR_WRITE	3	The media driver failed to write.
LF_ERR_FULL	4	No free sector is left.
LF_ERR_MEM	5	<code>pvPortMalloc()</code> returned NULL in <code>lf_open()</code> .
LF_ERR_NOTOPEN	6	A NULL handle, or the mode does not allow the operation.
LF_ERR_NOTFOUND	7	No record matches the name or expression.
LF_ERR_FNAME	8	The name is NULL or empty, or the record is not a regular file.

4 API reference

4.1 lf_init

```
void lf_init(void);
```

Weak hook called once at start up. The default body is empty; override it if a volume needs to be brought up before the first access.

4.2 lf_format

```
lf_err_t lf_format(uint32_t size, uint16_t devID, char *name);
```

Writes a fresh volume descriptor into sector 0 and releases every other sector. All data on the media is lost.

Parameter	Description
size	Media size in bytes. The resulting sector count is clamped to the sector count of the volume driver.
devID	Device identifier stored in the descriptor, LLFS_DEVID_EEPROM for the Rimer SBC part.
name	Volume name. Exactly LLFS_VOLNAME_LEN bytes are read as is, so a shorter buffer must not be passed; a string literal of at least 8 bytes is safe.

Returns LF_ERR_NONE, or the media error that stopped the format.

lf_error LF_ERR_WRITE when the media rejects a write.

Example

```
if (lf_format(LLFS_EEPROM_SIZE, LLFS_DEVID_EEPROM, "EEPROM") != LF_ERR_NONE)
    tprintf("format failed, lf_error = %d\n", lf_error);
```

4.3 lf_get_free

```
uint32_t lf_get_free(uint8_t volIndex);
```

Counts the released sectors and reports the space they represent.

Parameter	Description
volIndex	Volume index. Ignored, the EEPROM is the only volume at the moment.

Returns Free space in bytes, 0 if the media cannot be read.

If_error Set by the media driver.

Example

```
tprintf("%d bytes free\n", lf_get_free(0));
```

4.4 lf_get_fsize

```
uint32_t lf_get_fsize(char *name, uint16_t fPtr);
```

Walks the sector chain of a file and adds up the valid bytes. The file may be identified either by name or, when name is NULL or empty, by its first data sector.

Parameter	Description
name	File name or expression. When it is NULL or empty, fPtr is used instead.
fPtr	First data sector, that is lf_record_t::fptr, not the record position.

Returns File size in bytes, 0 if the file does not exist.

If_error LF_ERR_NOTFOUND when the name matches nothing.

Example

```
lf_record_t record;

tprintf("%d bytes\n", lf_get_fsize("conf.sys", 0));
if (lf_find_record("conf.sys", &record, 0)) // the same by sector
    tprintf("%d bytes\n", lf_get_fsize("", record.fptr));
```

4.5 lf_find_record

```
uint16_t lf_find_record(char *name, lf_record_t *record, uint8_t next);
```

Searches the index chain for a record whose name matches the expression and copies it out. Pass next as 0 to start a search and as 1 to continue it, which is how a whole volume is listed.

Parameter	Description
name	File name or expression, see section 3.2.
record	Receives a copy of the record found.
next	0 starts a new search, 1 continues the previous one.

Returns Record position, index sector in the high byte and record number in the low byte, or 0 when nothing matched.

If_error LF_ERR_NOTFOUND at the end of the search, LF_ERR_VOLUME if the media is not formatted.

Example

```
lf_record_t record;
char str[LLFS_FILENAME_LEN + 1];
uint8_t next = 0;

while (lf_find_record("", &record, next))
{
    next = 1;
    tprintf("%s\t%d\n", lf_rname_tostr(str, record.name),
            lf_get_fsize("", record.fptr));
}
```

One search at a time

The search cursor is a static variable inside `lf_find_record()`. A second search started before the first one has finished, in another task or inside the loop body, restarts the cursor.

4.6 lf_rname_tostr

```
char *lf_rname_tostr(char *destStr, char *name);
```

Converts a record name into a C string. This is the only safe way to read `lf_record_t::name`, which carries no terminator when all 12 characters are used.

Parameter	Description
destStr	Destination buffer, at least LLFS_FILENAME_LEN + 1 bytes.
name	Record name field, or any string to be truncated to 12 characters.

Returns destStr, always zero terminated.

Example

```
char str[LLFS_FILENAME_LEN + 1];
tprintf("%s\n", lf_rname_tostr(str, record.name));
```

4.7 lf_delete

```
void lf_delete(char *name);
```

Deletes every record matching the expression and releases the sectors of each file. Deleting with an expression removes all matches in one call.

Parameter	Description
name	File name or expression.

Returns Nothing.

If_error LF_ERR_NOTFOUND when nothing matched.

Example

```
lf_delete("conf.sys");    // one file
lf_delete("*.bak");      // every backup
lf_delete("");           // empty the volume
```

4.8 lf_open

```
lfile_t *lf_open(char *name, uint8_t mode);
```

Opens a file and allocates the handle and its sector cache from the FreeRTOS heap. A name longer than 12 characters is truncated. Directories, links and index records are refused.

Parameter	Description
name	File name, at most 12 characters are used.
mode	Any combination of the MODE_ constants, see section 3.4.

Returns An open file, or NULL. Every handle returned must be released with lf_close().

If_error LF_ERR_FNAME for an empty name or a record that is not a regular file, LF_ERR_MEM when the heap is exhausted, LF_ERR_NOTFOUND when the file does not exist and MODE_CREATE was not given.

Example

```
lfile_t *file;

if ((file = lf_open("conf.sys", MODE_CREATE | MODE_WRITE)) == NULL)
    return "cannot create conf.sys";
lf_write(file, &sysConf, sizeof(sysConf));
lf_close(file);
```

4.9 lf_close

```
void lf_close(lfile_t *file);
```

Writes the cached sector back if it was modified and releases the handle. Passing NULL is harmless. A handle must not be used after it has been closed.

Parameter	Description
file	The open file, or NULL.

Returns Nothing.

Example

```
lf_close(file);
```

4.10 lf_read

```
uint16_t lf_read(lfile_t *file, void *data, uint16_t size);
```

Reads from the current position and advances it. Reading stops at the end of the file, so a short count is the normal way to detect it.

Parameter	Description
file	A file opened with MODE_READ or MODE_WRITE.
data	Destination buffer.
size	Number of bytes wanted.

Returns Number of bytes actually read, which is less than size at the end of the file.

lf_error LF_ERR_NOTOPEN for a NULL handle or a mode that does not allow reading.

Example

```
uint8_t buff[64];
uint16_t count;

while ((count = lf_read(file, buff, sizeof(buff))) != 0)
    process(buff, count);
```

4.11 lf_write

```
uint16_t lf_write(lfile_t *file, void *data, uint16_t size);
```

Appends to the file, allocating sectors as it goes. A partially filled sector stays in the cache and reaches the media when the file is closed.

Parameter	Description
file	A file opened with MODE_WRITE.
data	Source buffer.
size	Number of bytes to write.

Returns Number of bytes written. A smaller value means the volume filled up; the bytes reported are safely stored.

If_error LF_ERR_NOTOPEN for a NULL handle or a read only one, LF_ERR_FULL when no sector is left.

Example

```
if (lf_write(file, buff, len) != len)
    tprintf("volume full, lf_error = %d\n", lf_error);
```

4.12 lf_gets

```
char *lf_gets(char *str, uint16_t size, lfile_t *file);
```

Reads one line, the newline included, and terminates it. A line longer than the buffer is returned in pieces; the last line of a file is returned even when it has no newline.

Parameter	Description
str	Destination buffer.
size	Buffer size, at most size-1 characters are stored.
file	A file opened for reading.

Returns str, or NULL at the end of the file.

If_error LF_ERR_NOTOPEN for a NULL handle.

Example

```
char line[64];

while (lf_gets(line, sizeof(line), file))
    shell_execute(line);
```

5 Worked examples

5.1 Format the volume on first start

```
lfile_t *file;

/* the first open tells whether the media carries a volume at all */
if ((file = lf_open(DEF_CONF_FILE_NAME, MODE_CREATE | MODE_WRITE)) == NULL)
{
    tprintf("unformatted media, formatting...");
    if (lf_format(LLFS_EEPROM_SIZE, LLFS_DEVID_EEPROM, "EEPROM") != LF_ERR_NONE)
        return "format failed";
    file = lf_open(DEF_CONF_FILE_NAME, MODE_CREATE | MODE_WRITE);
}
```

5.2 Save and load a configuration structure

```
/* save */
sysConf.checkSum = crc8((uint8_t *)&sysConf, sizeof(sysConf) - 1);
if ((file = lf_open(DEF_CONF_FILE_NAME, MODE_CREATE | MODE_WRITE)) != NULL)
{
    if (lf_write(file, &sysConf, sizeof(sysConf)) != sizeof(sysConf))
        tprintf("config not saved\n");
    lf_close(file);
}

/* load */
if ((file = lf_open(DEF_CONF_FILE_NAME, MODE_READ)) != NULL)
{
    _sysconf_t tmpConf;

    if (lf_read(file, &tmpConf, sizeof(tmpConf)) == sizeof(tmpConf) &&
        crc8((uint8_t *)&tmpConf, sizeof(tmpConf) - 1) == tmpConf.checkSum)
        memcpy(&sysConf, &tmpConf, sizeof(sysConf));
    lf_close(file);
}
```

There is no seek

A handle can only move forward. To rewrite a file that has just been read, close it and open it again with `MODE_WRITE`, which truncates it.

5.3 List the volume

```
lf_record_t record;
char str[LLFS_FILENAME_LEN + 1];
uint8_t next = 0;

while (lf_find_record("", &record, next))
{
    next = 1;
    tprintf("%s\t%d\t%c%c%c\n", lf_rname_tostr(str, record.name),
            lf_get_fsize("", record.fptr),
            record.attr & LLFS_ATTR_READ ? 'r' : '-',
            record.attr & LLFS_ATTR_WRITE ? 'w' : '-',
            record.attr & LLFS_ATTR_EXEC ? 'x' : '-');
}
tprintf("%d bytes free\n", lf_get_free(0));
```

5.4 Append to a log file

```
if ((file = lf_open("system.log", MODE_CREATE | MODE_WRITE | MODE_APPEND)) != NULL)
{
    lf_write(file, line, strlen(line));
    lf_close(file);
}
```

5.5 Run a script line by line

```
char line[80];

if ((file = lf_open("startup.rsh", MODE_READ)) != NULL)
{
    while (lf_gets(line, sizeof(line), file)) shell_execute(line);
    lf_close(file);
}
```

6 Porting LLFS to another media

6.1 The volume descriptor

A volume is a `volume_t`: a pointer to the physical description, five driver callbacks and one 256 byte scratch buffer that the module uses for index sectors.

```
typedef struct __attribute__((packed, aligned(4)))
{
    const lf_phy_t *phy;
    uint16_t (*write_sector)(uint8_t *data, uint16_t sector);
    uint16_t (*read_sector)(uint8_t *data, uint16_t sector);
    bool (*get_sector_next)(uint16_t *next, uint16_t sector);
    bool (*set_sector_next)(uint16_t *next, uint16_t sector);
    uint8_t (*init)(void *volume);
    uint8_t sData[LLFS_SECTOR_SIZE];
} volume_t;
```

6.2 Driver callbacks

Callback	Must do	Returns
<code>write_sector</code>	Store <code>LLFS_SECTOR_SIZE</code> bytes at sector $\times$ 256.	Bytes written, 0 on failure.
<code>read_sector</code>	Load <code>LLFS_SECTOR_SIZE</code> bytes from sector $\times$ 256.	Bytes read, 0 on failure.
<code>get_sector_next</code>	Read the two link bytes at sector $\times$ 256 + 254.	true on success.
<code>set_sector_next</code>	Write the two link bytes at sector $\times$ 256 + 254.	true on success.
<code>init</code>	Optional bring up of the media.	Driver defined.

Every callback must also set the global `lf_error`: `LF_ERR_NONE` on success, `LF_ERR_READ` or `LF_ERR_WRITE` on failure. The two link callbacks exist so that the free list can be updated without reading and rewriting a whole sector, which matters on media with a limited number of write cycles.

6.3 EEPROM driver

```
static const lf_phy_t phyEEPROM =
{
    .fsType    = LLFS_VOLUME_VALID,
    .fsTypeN   = (uint8_t)~LLFS_VOLUME_VALID,
    .devID     = LLFS_DEVID_EEPROM,
    .secCount  = LLFS_SECTOR_COUNT_EEPROM,
    .compress  = LLFS_COMPRESS_NONE,
    .ecc       = LLFS_ECC_NONE,
};

static volume_t eeepromIC =
{
    .phy = &phyEEPROM,
    .write_sector = eeeprom_write_sector,
    .read_sector  = eeeprom_read_sector,
    .get_sector_next = eeeprom_get_sector_next,
    .set_sector_next = eeeprom_set_sector_next,
};

volume_t *volumeEEPROM = &eeepromIC;
```

Page aligned writes

A sector starts at a multiple of 256 and the link bytes sit in the last page of the sector, so neither access crosses an EEPROM page boundary. A driver for a part with a different page size must keep that property, otherwise a page write wraps inside the page and corrupts data.

7 Limits and design notes

Limit	Detail
No seek	A handle moves forward only. Reopen the file to start again.
One task at a time	If <code>lf_error</code> , the scratch buffer inside <code>volume_t</code> and the search cursor of <code>lf_find_record()</code> are shared. Guard the module with a mutex if more than one task uses it.
Index sectors are not released	A sector that became an index sector stays one. The space is reused by later records but never returns to the free pool.
Sector 0xFF is reserved	It can only ever become the second index sector, because 0xFF is also the end of chain mark.
Names are truncated	Anything past 12 characters is dropped, silently and consistently, on create and on open.
Directories are not implemented	If <code>record_t::rptr</code> and the DIR and LINK attributes are reserved for a later version. The volume is flat.
One volume	<code>volume_t</code> supports several volumes but only the EEPROM is instantiated; <code>lf_get_free()</code> ignores its <code>volIndex</code> argument.
Power failure	The largest atomic operation is one sector write. An interrupted write truncates the file being written; the volume and every other file stay valid.

8 Verification

`llfs_unittest.c` runs the file system against a RAM image of the media on the development machine. It compiles `llfs.c` into its own translation unit, so the static internals are covered as well, and needs neither the target nor FreeRTOS.

```
cc -o llfs_unittest llfs_unittest.c -Illfs -Ikernel/include -Wall && ./llfs_unittest
```

The exit status is 0 when every check passes. Add `-v` to list the checks that pass as well as the ones that fail. Beside the API results the test verifies the structure of the whole image after each scenario:

- the index chain terminates and every index sector keeps its mark and a non zero count byte;
- no file chain reaches sector 0, sector 0xFF, a free sector or an address outside the volume;
- no sector belongs to two files;
- a sector nobody refers to is released, so nothing leaks.

The media driver of the test can be told to fail a chosen sector, which is how the error paths and the recovery from an interrupted write are exercised.

9 License

Copyright © 2025 Sergey Sanders, sergey@sesadesign.com

LLFS is free software released under the MIT License. The only condition is attribution: the copyright notice and the permission notice must stay with every copy or substantial portion of the software.

MIT License

Copyright (c) 2025 Sergey Sanders

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

The same notice heads every source file of the module: `llfs.h`, `llfs.c`, `llfs_vol_eeprom.h`, `llfs_vol_eeprom.c` and `llfs_unittest.c`.